

Instituto Tecnológico de Costa Rica

Escuela Ingeniería en Computadores

Curso: CE1104

Documentación técnica

Strenger_Tec_ Morse_Traslator

Estudiantes:

Felipe Sanabria Pacheco

Kevin Sanabria Quesada

Profesor:

Leonardo Araya Martínez

Semestre I 2026

Contents

1. Introducción.....	3
1.1. Propósito del documento	3
1.2. Descripción del proyecto.....	3
1.3. Objetivos y Alcance	3
2. Descripción General del Sistema.....	4
2.1. Perspectiva del Producto	4
2.2. Funcionalidades principales	4
2.3. Restricciones del sistema, Supuestos y Dependencias	4
3. Requisitos del Sistema.....	7
3.1. Requisitos Funcionales	7
3.2. Requisitos No Funcionales	7
4. Diseño de la Solución.....	8
4.1. Visión General de la Arquitectura	8
4.2. Descripción de Componentes	8
4.4. Diagrama de Flujo y Secuencia	9
1.1. Decisiones de Diseño y Justificación.....	10
2. Implementación y Validación de Calidad	12
2.1. Estructura del repositorio	12
2.2. Entorno de Desarrollo, Guía de instalación y Ejecución	12
2.3. Estrategias y Casos de Prueba.....	13
2.4. Resultados de pruebas y problemas encontrado.....	14
3.1. Herramientas de IA utilizadas	16
3.3. Declaración de Responsabilidad y Autoría.....	17
4. Conclusión.....	18

1. Introducción

Este proyecto se inspira en la serie Stranger Things y el uso del código Morse como medio de comunicación. Se desarrolla un sistema que permite a dos jugadores comunicarse mediante señales luminosas y sonoras.

1.1. Propósito del documento

Este documento describe el diseño, implementación y funcionamiento del sistema StrangerTEC Morse Traslator, dirigido a estudiantes y profesores.

1.2. Descripción del proyecto

El proyecto consiste en recrear el sistema de comunicación mediante luces utilizado por Will Byers y Jim Hopper, realizando un juego de 1vs1 en el que se enfrentan dos personas por ver quien puede acertar la palabra dada en código morse

1.3 Objetivos y Alcance

Objetivo general:

Desarrollar un sistema funcional de comunicación en código Morse.

Objetivos específicos:

Implementar entrada Morse, salida visual y sistema de evaluación.

2. Descripción General del Sistema

El sistema consta de dos partes una maqueta realizada en Arduino UNO y una computadora, donde desde el Arduino se presenta una palabra en código morse ya sea mediante luz y sonido, desde la maqueta se puede introducir código al igual que desde la computadora, al final gana quien sea más exacto para replicar la palabra

2.1. Perspectiva del Producto

El sistema opera tanto en Python como en Arduino por eso depende uno de otro, estos están relacionados uno con otro ya que Python genera una interfaz gráfica la cual da señal al Arduino para que envíe la palabra en código morse hacia la maqueta.

2.2. Funcionalidades principales

Enumera las funcionalidades de alto nivel del sistema (no detalle de implementación). Puede representarse como una lista o diagrama de casos de uso.

Ejemplo:

- [Proporción de palabra]: [el sistema brinda una palabra predefinida en código morse]
- [Replicación e palabra 1° parte]: [Desde la computadora un jugador puede intentar replicar la palabra recibida]
- [Replicación e palabra 2° parte]: [Desde la maqueta un jugador puede intentar replicar la palabra recibida]

2.3. Restricciones del sistema, Supuestos y Dependencias

Primero, se listan las limitaciones que condicionan el diseño o implementación: tecnológicas, regulatorias, de rendimiento, de tiempo, etc.

Ejemplo:

- [Aplicación usada]: El sistema solo funciona si es accionada desde la aplicación de Python conectada a la aplicación de arduino
- [tiempo]: no hay tiempo limite entre letra y letra pero si se tiene que cumplir con el tiempo que debe de durar cada intervalo es decir el toque corto debe de durara exactamente 1 segundo y el largo 3 segundos de lo contrario el sistema podría no reconocer la letra o reconocer otra distinta además de que se debe esperar a que el sistema registre una letra para poder empezar con la otra

Supuestos:

- [Conocimiento de código morse]: no todos los jugadores tendrán conocimiento del código morse y realmente que conjunto de intervalos de sonido y luz crear que carácter por ende la transferencia de la palabra en morse se realiza de manera lenta

Dependencias.

- [maqueta-computadora]: La parte de software del sistema debe de estar conectada a la parte de Hardware de este si no es así en sistema no funcionará
- [uso Python]: El sistema debe usara exclusivamente desde la aplicación de Python no se puede usar otra aplicación como seria Visual Studio Code

3. Requisitos del Sistema

3.1. Requisitos Funcionales

ID	Descripción	Prioridad	Criterio de Aceptación
RF-001	[Detectar puntos y rayas.]	Alta	[Para esto el sistema debe de reconocer según el tiempo que se presione el botón si es un punto o una raya]
RF-002	[Mostrar letras mediante LEDs.]	Alta	[El sistema debe reconocer que letra se quiere formar y mediante LEDs exponer que letra interpreto]
RF-003	[Evaluar mensajes.]	Alta	[El sistema debe de determinar que tan allegado fue la replicación a la palabra brindada]

3.2. Requisitos No Funcionales

Describe atributos de calidad del sistema: rendimiento, seguridad, disponibilidad, escalabilidad, usabilidad, etc. Usa identificadores RNF-XXX.

Ejemplo:

ID	Categoría	Descripción	Métrica / Valor Esperado
RNF-001	Rendimiento	El sistema debe responder a las solicitudes del usuario en tiempo aceptable.	Tiempo de respuesta < 3 segundos en condiciones normales.
RNF-002	Uso de hardware accesible.	El hardware, es decir las piezas utilizadas pueden ser adquiridas por cualquier persona	El hardware se puede encontrar y tiene un precio medio que hace que todos puedan acceder a él
RNF-003	Usabilidad	La interfaz debe ser intuitiva para usuarios no técnicos.	El 80% de los usuarios completan las tareas principales sin ayuda.

4. Diseño de la Solución

4.1. Visión General de la Arquitectura

El sistema utiliza una arquitectura simple basada en Arduino UNO, con entrada (botón), procesamiento (microcontrolador) y salida (LEDs y buzzer).

(para el diagrama de conexiones ver archivo adjunto: diagrama de conexiones)

4.2. Descripción de Componentes

Describe cada componente o módulo del sistema: qué hace, qué tecnología usa, qué interfaces expone y qué dependencias tiene.

Ejemplo:

Componente	Responsabilidad	Tecnología	Dependencias
Arduino UNO	Manda y recibe las señales para encender los leds y el buzzer	microcontrolador ATmega328P	Puerto USB, debe estar conectado para ser energizado
Buzzer	Sonido del código morse	magnética	Requieren fuente de alimentación constante de no más de 5V
Pulsador	Introducción de datos	Conmutación simple	Tiene que mantenerse pulsado para realizar la acción
LEDs	Salida de datos en forma de Luz	Semiconductores	La intensidad de luz depende de la intensidad de corriente
Protoboar	Sostener los componentes y dar pie a la maqueta	Conexión sin soldar	Conexión constante a fuente de alimentación

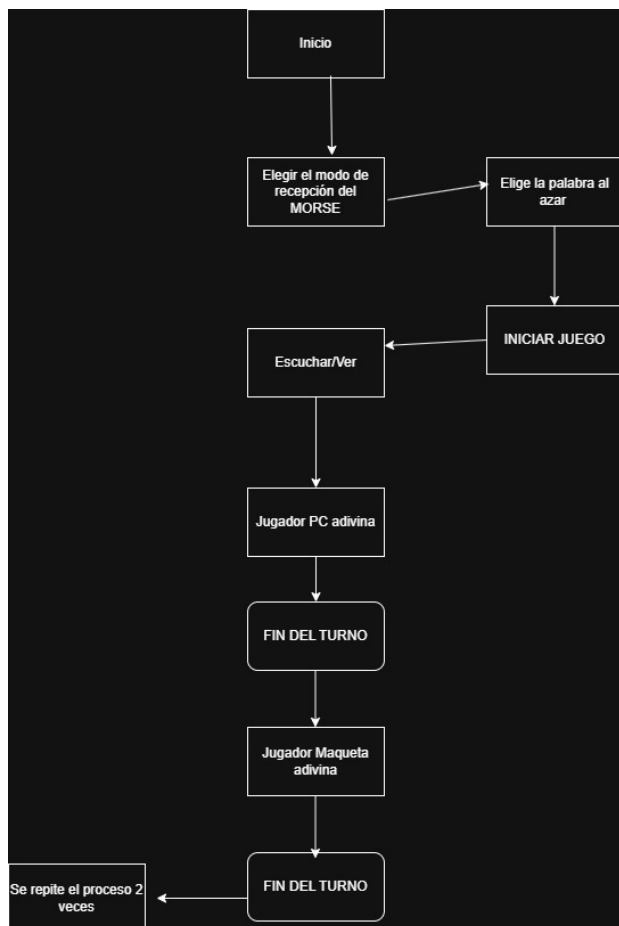
4.3. Stack Tecnológico

Lista todas las tecnologías, frameworks, librerías y herramientas utilizadas en el proyecto.

Ejemplo:

Categoría	Tecnología	Versión	Propósito
Lenguaje (Frontend)	Python	3.14	Lógica del lado cliente
Lenguaje (Backend)	Arduino	IDE 2.3.8	Programación de las funciones de Arduino(cuando encender LEDs y buzzer)

4.4. Diagrama de Flujo y Secuencia



1.1. Decisiones de Diseño y Justificación

Decisión	Alternativas Evaluadas	Opción Elegida	Justificación
Realización maqueta	Arduino Raspberry	Arduino UNO	No fue posible conseguir la Raspberry por ende se realizó en Arduino UNO
Interfaz Grafica	Tkinter	Tkinter	Es la herramienta sabíamos utilizar y encontramos más eficiente para realizar esta tarea

2. Implementación y Validación de Calidad

Describe la estructura del repositorio de código, el entorno de desarrollo, el proceso de construcción del proyecto y los pasos necesarios para su despliegue. Además para la validación de calidad se documenta la estrategia de pruebas del sistema, los resultados obtenidos, los problemas encontrados que tuvieron solución y los que no se pudieron resolver.

Se fundamenta en los estándares ISO/IEC 25010 (calidad de software), las guías de pruebas de Microsoft y la metodología de testing de Google SRE.

2.1. Estructura del repositorio

Describe la organización de carpetas y archivos del proyecto. Explica qué contiene cada directorio principal.

Ejemplo:

```
/proyecto-raiz  /src          → Código fuente principal  /tests          → Pruebas
unitarias e integración  /docs          → Documentación del proyecto  /config          →
Archivos de configuración  /scripts          → Scripts de automatización  README.md
→ Descripción general y guía de inicio rápido
```

2.2. Entorno de Desarrollo, Guía de instalación y Ejecución

Lista los prerequisites de software necesarios para ejecutar el proyecto en un entorno local de desarrollo. Además, se proporcionan instrucciones paso a paso para que cualquier persona pueda ejecutar el proyecto desde cero. Incluye comandos exactos.

Ejemplo:

- Entorno:
 - Sistema Operativo: [Windows 10+, macOS 12+, Ubuntu 20.04+]
 - Lenguaje / Runtime: [Ej. Node.js 18+ / Python 3.10+]

- Guía de instalación y ejecución:

```
0 # 1. Clonar el repositorio git clone
  https://github.com/[usuario]/[repositorio].git cd [repositorio] # 2.
  Instalar dependencias [npm install / pip install -r requirements.txt] #
  3. Configurar variables de entorno cp .env.example .env # Editar .env con
  los valores correspondientes # 4. Inicializar la base de datos [comando de
  migración] # 5. Ejecutar el servidor [npm start / python app.py]
```

2.3. Estrategias y Casos de Prueba

Describe la estrategia general de pruebas adoptada: tipos de prueba, herramientas utilizadas, cobertura, objetivo y responsables. Adicionalmente, se definen y documentan los casos de prueba más representativos con identificador, precondiciones, pasos, resultado esperado y resultado obtenido.

Ejemplo:

- Estrategias de pruebas:

Tipo de Prueba	Descripción	Herramienta	Cobertura Objetivo
Pruebas Unitarias	Verificación de funciones y métodos individuales de forma aislada en la interfaz grafica.	Python	>= 70% del código
Pruebas Unitarias	Verificación de funciones y métodos individuales de forma aislada en la interfaz grafica.	Arduino	Revisa que las conexiones se hayan realizado de manera correcta
Pruebas Funcionales	Validación del comportamiento del sistema desde la perspectiva del usuario final.	Pyhton, Arduino y maqueta	Casos de uso principales

- Casos de prueba:

Prueba	Interfaz / Parte	Cómo se probó	Problema encontrado	Solución / Mejora aplicada
1	Interfaz gráfica en Python	Ejecución directa del código y pruebas visuales de la interfaz.	Errores de espaciado y mala posición de elementos.	Ajuste de márgenes, alineación y estructura del código.
2	Interfaz gráfica en Python	Iteraciones progresivas corrigiendo errores anteriores.	Detalles de diseño y funcionamiento incompletos.	Refinamiento continuo hasta lograr una interfaz funcional.
3	Código en Arduino (independiente)	Pruebas sin conexión a la maqueta.	Funcionamiento correcto de forma aislada.	Validación inicial del código antes de integración.
4	Integración Arduino + maqueta	Conexión física y ejecución del sistema completo.	LEDs y buzzer se activaban aleatoriamente.	Corrección en la asignación y coordinación de puertos.
5	Sistema de código Morse	Pruebas del tiempo entre puntos y rayas.	Dificultad para diferenciar letras por tiempos muy cortos.	Ajuste en la duración de señales y pausas.
6	Sistema final integrado	Prueba completa del proyecto.	Funcionamiento poco claro al inicio.	Optimización general hasta alcanzar el modelo final.

2.4. Resultados de pruebas y problemas encontrado

- Resultados:

Se realizaron varias pruebas hasta que cada parte del sistema funciono correctamente en conjunto

Problemas encontrados:

- Problemas Solucionados: Como se realizo la programación aparte de la maqueta la primera prueba nada funcionaba ya que los pines de control utilizados no eran los mismos de la maqueta reorganizándolos todo funcionó , además s tuvo que colocar otra protoboar ya que al sobre cargar una sola protoboar hace que haga cortocircuito y queden LEDs encendido cuando deberían de apagarse.

3. Declaración de uso de Inteligencia Artificial y herramientas

Principio de Transparencia Académica

El uso de herramientas de Inteligencia Artificial en proyectos académicos requiere declaración explícita y responsable. Esta sección documenta de forma honesta cómo, cuándo y para qué se utilizaron herramientas de IA en el desarrollo de este proyecto, siguiendo los principios de integridad académica

3.1. Herramientas de IA utilizadas

Herramienta de IA	Tipo	Período de Uso	Propósito
ChatGPT / GPT-4	Modelo de lenguaje (LLM)	[abril/2026]	Generación de ideas, aportes al código, resolución de dudas técnicas.

3.2. Descripción del Uso de Inteligencia Artificial

- Generación y asistencia en código

3.3. Declaración de Responsabilidad y Autoría

Esta declaración debe ser redactada y firmada por todos los integrantes del equipo. Afirma que el trabajo final es producto del esfuerzo intelectual del equipo y que la IA fue utilizada como herramienta de apoyo.

Ejemplo:

DECLARACIÓN DE AUTORÍA Y RESPONSABILIDAD

Los suscritos, integrantes del equipo de trabajo del proyecto "[Strenger_Tec_ Morse_Traslator]", declaramos que:

1. El diseño, análisis, desarrollo y conclusiones del presente proyecto son resultado de nuestro trabajo intelectual y esfuerzo como equipo.
2. Las herramientas de Inteligencia Artificial fueron utilizadas exclusivamente como apoyo en tareas específicas descritas en este capítulo, no como sustituto de nuestro trabajo.
3. Todo el contenido generado por IA fue revisado, comprendido, validado y, cuando fue necesario, corregido por los integrantes del equipo.
4. Asumimos plena responsabilidad académica sobre el contenido total de este proyecto y su documentación.
5. El uso de IA se realizó conforme a las políticas de la institución educativa y los lineamientos del docente responsable.

4. Conclusión

Si se logró completar los objetivos propuestos por el docente de manera satisfactoria completando con éxito la elaboración del proyecto: “Stranger_Tec”, al conseguir la comunicación Arduino computadora, y hacer que se pueda realizar un 1vs1 entre la computadora y la maqueta realizada con Arduino.

Si bien hubo algo de complicación a la hora de conectar la interfaz gráfica con la programación de Arduino, fue un reto que se pudo superar al lograr esa conexión para que el Arduino pudiera emitir código morse tanto en energía lumínica como en energía sónica, y recibiendo de manera correcta replicación de código morse a través de un botón.

Dentro de los conocimientos técnicos adquiridos está el cómo conectar una interfaz gráfica realizada en Python con una programación de Arduino y hacer que sean dependientes entre sí, además de volver a aplicar conocimientos sobre conexiones de Arduino y todo lo que se puede lograr con este, respecto al trabajo en equipo se logró trabajar en conjunto de manera eficiente, en conjunto para lograr la finalización de este proyecto